DARSHAN PRAJAPATI
MCA-1
ROLL NO: 24
SUBMISSION DATE: 16/09/2025

OOCP PRACTICAL ASSIGNMENT-2

// 1) WAP to use binary operator + add two object of class Numbers having num1

// and num2 as its data members and display result.

```
#include <iostream>
using namespace std;
class Numbers
{
   int data1;
   int data2;

public:
   Numbers(int d1, int d2) : data1(d1), data2(d2) {}
   Numbers operator+(Numbers &temp)
   {
      return Numbers(this->data1 + temp.data1, this->data2 + temp.data2);
   }
   void display()
   {
      cout << "Sum of numbers " << data1 << "+" << data2 << "=" << data1 + data2 << endl;
   }
};
int main()
{
   Numbers n1(5, 8), n2(5, 10);
   Numbers n3 = n1 + n2;
   n1.display();
   n2.display();
   n3.display();

   return 0;
}
```

// Output

// Sum of numbers 5+8=13

// Sum of numbers 5+10=15

// Sum of numbers 10+18=28

// 2) WAP to overload operator * which multiply a number to each element of an

// array within a class arrayContainer and display the result.

```
#include <iostream>
using namespace std;

class arrayContainer
{
private:
   int arr[7] = {1, 2, 3, 4, 5, 8, 10};

public:
   // Default constructor
   arrayContainer() {}

   // Overloading the '*' operator
   arrayContainer operator*(int num)
   {
      arrayContainer arr2;
      for (int i = 0; i < 7; i++)
      {
         arr2.arr[i] = arr[i] * num;
      }
      return arr2;
   }

   void display()
   {
      for (int i = 0; i < 7; i++)
      {
         cout << arr[i] << " ";
      }
```

```
        cout << endl;
    }
};

int main()
{
    arrayContainer a1, a2;

    cout << "Original array:" << endl;
    a1.display();

    a2 = a1 * 5; // Multiply each element by 5

    cout << "Array after multiplication:" << endl;
    a2.display();

    return 0;
}
```

// Output

```
// Original array:
// 1 2 3 4 5 8 10
// Array after multiplication:
// 5 10 15 20 25 40 50
```

// 3) WAP to Overload the *, +,-, ==, ! = and = operators for the complex class.

```
#include <iostream>
using namespace std;

class complex
{
private:
    float real;
    float imag;

public:
```

```
// Constructor
complex(float r = 0, float i = 0)
{
   real = r;
   imag = i;
}

// Overloading '+' operator
complex operator+(const complex &obj)
{
   return complex(real + obj.real, imag + obj.imag);
}

// Overloading '-' operator
complex operator-(const complex &obj)
{
   return complex(real - obj.real, imag - obj.imag);
}

// Overloading '*' operator
complex operator*(const complex &obj)
{
   float r = real * obj.real - imag * obj.imag;
   float i = real * obj.imag + imag * obj.real;
   return complex(r, i);
}

// Overloading '==' operator
bool operator==(const complex &obj)
{
   return (real == obj.real && imag == obj.imag);
}

// Overloading '!=' operator
bool operator!=(const complex &obj)
{
```

```
        return !(*this == obj);
    }

    // Overloading '=' operator
    complex &operator=(const complex &obj)
    {
        if (this != &obj)
        { // Avoid self-assignment
            real = obj.real;
            imag = obj.imag;
        }
        return *this;
    }

    // Function to display complex number
    void display()
    {
        cout << real;
        if (imag >= 0)
            cout << " + " << imag << "i";
        else
            cout << " - " << -imag << "i";
        cout << endl;
    }
};

int main()
{
    complex c1(3, 4), c2(1, 2), c3;

    cout << "First complex number: ";
    c1.display();

    cout << "Second complex number: ";
    c2.display();
```

```
    // Addition
    c3 = c1 + c2;
    cout << "Addition result: ";
    c3.display();

    // Subtraction
    c3 = c1 - c2;
    cout << "Subtraction result: ";
    c3.display();

    // Multiplication
    c3 = c1 * c2;
    cout << "Multiplication result: ";
    c3.display();

    // Assignment
    c3 = c1;
    cout << "After assignment, c3 = ";
    c3.display();

    // Equality
    if (c1 == c3)
        cout << "c1 and c3 are equal." << endl;
    else
        cout << "c1 and c3 are not equal." << endl;

    // Inequality
    if (c1 != c2)
        cout << "c1 and c2 are not equal." << endl;
    else
        cout << "c1 and c2 are equal." << endl;

    return 0;
}
```

// output

```
// First complex number: 3 + 4i
// Second complex number: 1 + 2i
// Addition result: 4 + 6i
// Subtraction result: 2 + 2i
// Multiplication result: -5 + 10i
// After assignment, c3 = 3 + 4i
// c1 and c3 are equal.
// c1 and c2 are not equal.
```

// 4) WAP to define an object m1 of matrix class, use m1<<cout.

```
#include <iostream>
using namespace std;

class matrix
{
private:
   int mat[2][2];

public:
   // Constructor to initialize matrix elements
   matrix(int a = 0, int b = 0, int c = 0, int d = 0)
   {
      mat[0][0] = a;
      mat[0][1] = b;
      mat[1][0] = c;
      mat[1][1] = d;
   }

   // Friend function to overload << operator
   friend ostream &operator<<(ostream &out, const matrix &m);
};

// Definition of overloaded << operator
ostream &operator<<(ostream &out, const matrix &m)
{
```

```
    out << m.mat[0][0] << " " << m.mat[0][1] << endl;
    out << m.mat[1][0] << " " << m.mat[1][1] << endl;
    return out;
}

int main()
{
    matrix m1(1, 2, 3, 4);

    cout << "The matrix m1 is:" << endl;
    cout << m1;

    return 0;
}
```

// output:

```
// The matrix m1 is:
// 1 2
// 3 4
```

// 5) WAP to define a matrix class and overload the * operator to multiply a number

// with matrix (Example: 5*Matrix should be possible).

```
#include <iostream>
using namespace std;

class matrix
{
    int mat[2][2];

public:
    // Constructor to initialize matrix elements
    matrix(int a = 0, int b = 0, int c = 0, int d = 0)
    {
        mat[0][0] = a;
```

```
    mat[0][1] = b;
    mat[1][0] = c;
    mat[1][1] = d;
}

// Overloading '*' operator for matrix * scalar
matrix operator*(int multiplier)
{
    matrix result;
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            result.mat[i][j] = mat[i][j] * multiplier;
        }
    }
    return result;
}

// Overloading '*' operator for scalar * matrix
friend matrix operator*(int multiplier, matrix &obj);

// Overloading '<<' operator to display matrix
friend ostream &operator<<(ostream &out, const matrix &obj)
{
    out << "Matrix values:" << endl;
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            out << obj.mat[i][j] << "\t";
        }
        out << endl;
    }
    return out;
}
```

```
};

// Definition of scalar * matrix
matrix operator*(int multiplier, matrix &obj)
{
    matrix result;
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            result.mat[i][j] = obj.mat[i][j] * multiplier;
        }
    }
    return result;
}

int main()
{
    matrix m1(2, 4, 8, 10), m2;

    cout << "Original matrix m1:" << endl;
    cout << m1;

    // matrix * scalar
    m2 = m1 * 5;
    cout << "After multiplying by 5:" << endl;
    cout << m2;

    // scalar * matrix
    m2 = 10 * m1;
    cout << "After multiplying by 10:" << endl;
    cout << m2;

    return 0;
}
```

// output:

// Original matrix m1:

// Matrix values:

// 2 4

// 8 10

// After multiplying by 5:

// Matrix values:

// 10 20

// 40 50

// After multiplying by 10:

// Matrix values:

// 20 40

// 80 100

// 6) WAP to define a class Date with properties int month; int day; int year;

// overload the following operators.

// 5.1) + operator [a+b] (a is of date type and b is an integer),

// use the assumption that all years have 360 days and

// months 30 days.

// 5.2) – operator [a-b(same as above)]

// 5.3) = operator

// 5.4) <,<=,>,>=

// 5.5) ++,--[post and pre both]

```
#include <iostream>
using namespace std;

class Date
{
   int month;
   int day;
   int year;

public:
   // constructor
```

```
Date(int d = 1, int m = 1, int y = 1) : day(d), month(m), year(y) {};

// normalize the date
void normalize()
{
    while (day > 30)
    {
        day -= 30;
        month++;
    }

    while (day <= 0)
    {
        day += 30;
        month--;
    }
    while (month > 12)
    {
        month -= 12;
        year++;
    }

    while (month <= 0)
    {
        month += 12;
        year--;
    }
}

// Overload '+' operator (Date + int days)
Date operator+(int days)
{
    Date temp = *this;
    temp.day += days;
    temp.normalize();
    return temp;
```

```
}

// Overload '-' operator (Date + int days)
Date operator-(int days)
{
   Date temp = *this;
   temp.day -= days;
   temp.normalize();
   return temp;
}

// Overload '=' operator (assignment)
Date &operator=(const Date &other)
{
   if (this != &other)
   {
      this->day = other.day;
      this->month = other.month;
      this->year = other.year;
   }
   return *this;
}

// convert date into totalDays
int totalDays() const
{
   return year * 360 + (month - 1) * 30 + (day - 1);
}

// overload <,<=,>,>= operators
// overload ' < '
bool operator<(const Date &other)
{
   return totalDays() < other.totalDays();
}
// overload ' <= '
```

```
bool operator<=(const Date &other)
{
    return totalDays() <= other.totalDays();
}
// overload ' > '
bool operator>(const Date &other)
{
    return totalDays() > other.totalDays();
}
// overload ' >= '
bool operator>=(const Date &other)
{
    return totalDays() >= other.totalDays();
}

// overload ++,--[post and pre both] operators
// Pre-increment (++date)
Date &operator++()
{
    day++;
    normalize();
    return *this;
}
// Post-increment (date++)
Date operator++(int)
{
    Date temp = *this;
    day++;
    normalize();
    return temp;
}
// Pre-decrement (--date)
Date &operator--()
{
    day--;
    normalize();
```

```
        return *this;
    }
    // Post-decrement (date--)
    Date operator--(int)
    {
        Date temp = *this;
        day--;
        normalize();
        return temp;
    }

    // display
    void display()
    {
        cout << "\tDay: " << day << "\t Month: " << month << "\tYear: " << year << endl;
    }
};
int main()
{
    Date a(12, 30, 2023);
    Date b;

    cout << "Original date a: ";
    a.display();

    // Adding 45 days
    b = a + 45;
    cout << "After adding 45 days: ";
    b.display();

    // Subtracting 40 days
    b = a - 40;
    cout << "After subtracting 40 days: ";
    b.display();

    // Assignment
```

```
    b = a;
    cout << "After assignment, b = ";
    b.display();

    // Comparison
    Date c(1, 1, 2024);
    cout << "Comparing a and c:" << endl;
    cout << "a < c: " << (a < c) << endl;
    cout << "a <= c: " << (a <= c) << endl;
    cout << "a > c: " << (a > c) << endl;
    cout << "a >= c: " << (a >= c) << endl;

    // Pre-increment
    ++a;
    cout << "After pre-increment ++a: ";
    a.display();

    // Post-increment
    a++;
    cout << "After post-increment a++: ";
    a.display();

    // Pre-decrement
    --a;
    cout << "After pre-decrement --a: ";
    a.display();

    // Post-decrement
    a--;
    cout << "After post-decrement a--: ";
    a.display();

    return 0;
}
```

// output

// Original date a: Day: 12 Month: 30 Year: 2023

// After adding 45 days: Day: 27 Month: 7 Year: 2025

// After subtracting 40 days: Day: 2 Month: 5 Year: 2025

// After assignment, b = Day: 12 Month: 30 Year: 2023

// Comparing a and c:

// a < c: 0

// a <= c: 0

// a > c: 1

// a >= c: 1

// After pre-increment ++a: Day: 13 Month: 6 Year: 2025

// After post-increment a++: Day: 14 Month: 6 Year: 2025

// After pre-decrement --a: Day: 13 Month: 6 Year: 2025

// After post-decrement a--: Day: 12 Month: 6 Year: 2025

// 7) WAP to define a class Time with properties int hour; int minute; int second;

// overload the following operators.

// 6.1) + operator [a+b] (a is of time type and b is an integer)

// 6.2) – operator [a-b(same as above)]

// 6.3) = operator

// 6.4) <,<=,>,>=

// 6.5) ++,--[post and pre both]

```
#include <iostream>
using namespace std;
class Time
{
   int hour;
   int minute;
   int second;

public:
   Time(int h = 0, int m = 0, int s = 0) : hour(h), minute(m), second(s) {};
```

```
// Normalize the time
void normalize()
{
   while (second >= 60)
   {
      second -= 60;
      minute++;
   }
   while (second < 0)
   {
      second += 60;
      minute--;
   }
   while (minute >= 60)
   {
      minute -= 60;
      hour++;
   }
   while (minute < 0)
   {
      minute += 60;
      hour--;
   }
   while (hour >= 24)
   {
      hour -= 24;
   }
   while (hour < 0)
   {
      hour += 24;
   }
}

// operator ' + '
Time operator+(int secs)
{
```

```
    Time temp = *this;
    temp.second += secs;
    temp.normalize();
    return temp;
}
// operator ' - '
Time operator-(int secs)
{
    Time temp = *this;
    temp.second -= secs;
    temp.normalize();
    return temp;
}
// operator ' = '
Time &operator=(const Time &temp)
{

    if (this != &temp)
    {
        this->hour = temp.hour;
        this->minute = temp.minute;
        this->second = temp.second;
    }
    return *this;
}

// Convert total time to seconds for comparison
int totalSeconds() const
{
    return hour * 3600 + minute * 60 + second;
}

// Comparison operators
bool operator<(const Time &t) const { return totalSeconds() < t.totalSeconds(); }
bool operator<=(const Time &t) const { return totalSeconds() <= t.totalSeconds(); }
bool operator>(const Time &t) const { return totalSeconds() > t.totalSeconds(); }
```

```
bool operator>=(const Time &t) const { return totalSeconds() >= t.totalSeconds(); }
// Pre-increment (++t) adds 1 second
Time &operator++()
{
   second++;
   normalize();
   return *this;
}

// Post-increment (t++) adds 1 second
Time operator++(int)
{
   Time temp = *this;
   second++;
   normalize();
   return temp;
}

// Pre-decrement (--t) subtracts 1 second
Time &operator--()
{
   second--;
   normalize();
   return *this;
}

// Post-decrement (t--) subtracts 1 second
Time operator--(int)
{
   Time temp = *this;
   second--;
   normalize();
   return temp;
}
// display time
void
```

```
    display()
    {
        cout << (hour < 10 ? "0" : "") << hour << ":"
            << (minute < 10 ? "0" : "") << minute << ":"
            << (second < 10 ? "0" : "") << second << endl;
    }
};

int main()
{
    Time t1(23, 59, 50), t2;
    cout << "Original time t1: ";
    t1.display();

    // +
    t2 = t1 + 15;
    cout << "After adding 15 seconds:";
    t2.display();

    // +
    t2 = t1 - 75;
    cout << "After subtracting 75 seconds:";
    t2.display();

    // =
    t2 = t1;
    cout << "After assignment, t2 = ";
    t2.display();

    // Comparison
    Time t3(22, 59, 59);
    cout << "t1 < t3: " << (t1 < t3) << endl;
    cout << "t1 <= t3: " << (t1 <= t3) << endl;
    cout << "t1 > t3: " << (t1 > t3) << endl;
    cout << "t1 >= t3: " << (t1 >= t3) << endl;
```

```
    // Pre-increment
    ++t1;
    cout << "After pre-increment ++t1: ";
    t1.display();

    // Post-increment
    t1++;
    cout << "After post-increment t1++: ";
    t1.display();

    // Pre-decrement
    --t1;
    cout << "After pre-decrement --t1: ";
    t1.display();

    // Post-decrement
    t1--;
    cout << "After post-decrement t1--: ";
    t1.display();
    return 0;
}
```

// output

```
// Original time t1: 23:59:50
// After adding 15 seconds:00:00:05
// After subtracting 75 seconds:23:58:35
// After assignment, t2 = 23:59:50
// t1 < t3: 0
// t1 <= t3: 0
// t1 > t3: 1
// t1 >= t3: 1
// After pre-increment ++t1: 23:59:51
// After post-increment t1++: 23:59:52
// After pre-decrement --t1: 23:59:51
// After post-decrement t1--: 23:59:50
```

// 8) Write a menu driven program that can perform the following functions on

// strings. (Use overloaded operators where possible).(Do not use predefined

// string function or class.)

// 1. Compare two strings for equality (== operator)

// 2. Check whether first string is smaller than the second (<= operator)

// 3. Copy the string to another

// 4. Extract a character from the string (Overload [])

// 5. Reverse the string

// 6. Concatenate two strings (+ operator)

```
#include <iostream>
using namespace std;

class MyString
{
   int length;
   char str[100];

public:
   // Default constructor
   MyString()
   {
      length = 0;
      str[0] = '\0';
   }

   // Constructor to initialize from char array
   MyString(const char s[])
   {
      length = 0;
      while (s[length] != '\0')
      {
         str[length] = s[length];
         length++;
      }
      str[length] = '\0';
```

```
}

// display string
void display()
{
    for (int i = 0; i < length; i++)
    {
        cout << str[i];
    }
    cout << endl;
}

//  1. Compare two strings for equality (== operator)
// Overload '==' to compare two strings
bool operator==(const MyString &other)
{
    if (length != other.length)
        return false;
    for (int i = 0; i < length; i++)
    {
        if (str[i] != other.str[i])
            return false;
    }
    return true;
}

// 2. Check whether first string is smaller than the second (<= operator)
// Overload '<=' to check if first string is smaller or equal lexicographically
bool operator<=(const MyString &other)
{
    int minLen = (length < other.length) ? length : other.length;
    for (int i = 0; i < minLen; i++)
    {
        if (str[i] < other.str[i])
            return true;
        else if (str[i] > other.str[i])
```

```
        return false;
    }
    return length <= other.length;
}

// 3. Copy the string to another
// Overload '=' to copy one string to another
MyString &operator=(const MyString &other)
{
    length = other.length;
    for (int i = 0; i < length; i++)
    {
        str[i] = other.str[i];
    }
    str[length] = '\0';
    return *this;
}

// 4. Extract a character from the string (Overload [])
// Overload '[]' to extract character at position
char operator[](int index)
{
    if (index >= 0 && index < length)
        return str[index];
    else
    {
        cout << "Index out of bounds!" << endl;
        return '\0';
    }
}

// 5. Reverse the string
// Reverse the string
void reverse()
{
    for (int i = 0; i < length / 2; i++)
```

```
        {
            char temp = str[i];
            str[i] = str[length - i - 1];
            str[length - i - 1] = temp;
        }
    }

    // 6. Concatenate two strings (+ operator)
    // Overload '+' to concatenate two strings
    MyString operator+(const MyString &other)
    {
        MyString result;
        result.length = 0;
        // copy first string
        for (int i = 0; i < length; i++)
        {
            result.str[result.length++] = str[i];
        }
        // copy second string
        for (int i = 0; i < other.length; i++)
        {
            result.str[result.length++] = other.str[i];
        }
        // return concatenated string
        return result;
    }
};

int main()
{
    MyString s1, s2, s3;
    char temp[100];
    int choice;
    cout << "Enter string 1: ";
    cin >> temp;
    s1 = MyString(temp);
```

```
cout << "Enter string 2: ";
cin >> temp;
s2 = MyString(temp);
do
{
    cout << "\n--- Menu ---\n";
    cout << "1. Compare two strings (==)\n";
    cout << "2. Check if first string <= second string\n";
    cout << "3. Copy first string to another\n";
    cout << "4. Extract a character from first string ([])\n";
    cout << "5. Reverse the first string\n";
    cout << "6. Concatenate two strings (+)\n";
    cout << "0. Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;
    switch (choice)
    {
    case 1:
        if (s1 == s2)
            cout << "Strings are equal.\n";
        else
            cout << "Strings are not equal.\n";
        break;
    case 2:
        if (s1 <= s2)
            cout << "First string is smaller or equal.\n";
        else
            cout << "First string is larger.\n";
        break;
    case 3:
        s3 = s1;
        cout << "After copying, new string is: ";
        s3.display();
        break;
    case 4:
```

```
        int index;
        cout << "Enter index to extract from first string: ";
        cin >> index;
        cout << "Character at index " << index << ": " << s1[index] << endl;
        break;
      case 5:
        s1.reverse();
        cout << "First string after reversing: ";
        s1.display();
        break;
      case 6:
        s3 = s1 + s2;
        cout << "After concatenation: ";
        s3.display();
        break;
      case 0:
        cout << "Exiting....\n";
        break;
      default:
        cout << "Invalid choice! \n";
      }
    }

    while (choice != 0);

    return 0;
}
```

// output:

```
// Enter string 1 : apple
// Enter string 2 : banana

//---Menu-- -
// 1. Compare two strings(==) 2. Check if first string <= second string
// 3. Copy first string to another
// 4. Extract a character from first string([]) 5. Reverse the first string
```

// 6. Concatenate two strings(+)

// 0. Exit

// Enter your choice : 1 Strings are not equal.

//---Menu-- -

// 1. Compare two strings(==) 2. Check if first string <= second string

// 3. Copy first string to another

// 4. Extract a character from first string([]) 5. Reverse the first string

// 6. Concatenate two strings(+)

// 0. Exit

// Enter your choice : 2 First string is smaller or equal.

//---Menu-- -

// 1. Compare two strings(==) 2. Check if first string <= second string

// 3. Copy first string to another

// 4. Extract a character from first string([]) 5. Reverse the first string

// 6. Concatenate two strings(+)

// 0. Exit

// Enter your choice : 3 After copying,

// new string is : apple

//---Menu-- -

// 1. Compare two strings(==) 2. Check if first string <= second string

// 3. Copy first string to another

// 4. Extract a character from first string([]) 5. Reverse the first string

// 6. Concatenate two strings(+)

// 0. Exit

// Enter your choice : 4

// Enter index to extract from first string : 3

// Character at index 3 : l

//---Menu-- -

// 1. Compare two strings(==) 2. Check if first string <= second string

// 3. Copy first string to another

// 4. Extract a character from first string([]) 5. Reverse the first string

// 6. Concatenate two strings(+)

// 0. Exit

// your choice : 5 First string after reversing : elppa

//---Menu-- -

// 1. Compare two strings(==) 2. Check if first string <= second string

// 3. Copy first string to another

// 4. Extract a character from first string([]) 5. Reverse the first string

// 6. Concatenate two strings(+)

// 0. Exit

// Enter your choice : 6 After concatenation : elppabanana

//---Menu-- -

// 1. Compare two strings(==) 2. Check if first string <= second string

// 3. Copy first string to another

// 4. Extract a character from first string([]) 5. Reverse the first string

// 6. Concatenate two strings(+)

// 0. Exit

// Enter your choice : 0 Exiting....

// 9) WAP to Overload the New and Delete for Stack Class.

```
#include <iostream>
#include <cstdlib> // for malloc and free
using namespace std;

class Stack
{
   int top;
   int capacity;
   int *arr;

public:
   // Constructor
   Stack(int size = 10)
   {
      capacity = size;
```

```
    top = -1;
    arr = new int[capacity];
    cout << "Stack created with capacity " << capacity << endl;
}

// Destructor
~Stack()
{
    delete[] arr;
    cout << "Stack destroyed\n";
}

// Push operation
void push(int value)
{
    if (top >= capacity - 1)
    {
        cout << "Stack Overflow!\n";
        return;
    }
    arr[++top] = value;
}

// Pop operation
int pop()
{
    if (top < 0)
    {
        cout << "Stack Underflow!\n";
        return -1;
    }
    return arr[top--];
}

// Display the stack
void display()
```

```
    {
        if (top < 0)
        {
            cout << "Stack is empty\n";
            return;
        }
        cout << "Stack elements: ";
        for (int i = 0; i <= top; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    // Overloading new operator
    void *operator new(size_t size)
    {
        cout << "Overloading new for Stack, size = " << size << endl;
        void *ptr = malloc(size);
        if (!ptr)
        {
            throw bad_alloc();
        }
        return ptr;
    }

    // Overloading delete operator
    void operator delete(void *ptr)
    {
        cout << "Overloading delete for Stack\n";
        free(ptr);
    }
};

int main()
{
```

```
    // Using overloaded new and delete
    Stack *s = new Stack(5);

    s->push(10);
    s->push(20);
    s->push(30);
    s->display();

    cout << "Popped: " << s->pop() << endl;
    s->display();

    delete s; // Calls overloaded delete

    return 0;
}
```

// Output:

```
// Overloading new for Stack, size = 12
// Stack created with capacity 5
// Stack elements: 10 20 30
// Popped: 30
// Stack elements: 10 20
// Stack destroyed
// Overloading delete for Stack
```

// 10) Write a template function to make sum of two numbers.

```
#include <iostream>
using namespace std;

// Template function to add two numbers
template <typename T>
T sum(T a, T b)
{
    return a + b;
```

```
}

int main()
{
   // Sum of integers
   int x = 5, y = 10;
   cout << "Sum of integers: " << sum(x, y) << endl;

   // Sum of floats
   float p = 2.5f, q = 4.5f;
   cout << "Sum of floats: " << sum(p, q) << endl;

   // Sum of doubles
   double u = 3.14, v = 1.86;
   cout << "Sum of doubles: " << sum(u, v) << endl;

   return 0;
}
```

// output

```
// Sum of integers: 15
// Sum of floats: 7
// Sum of doubles: 5
```

// 11) Write a program to generate templates function for swapping values of
// variables and show its use with integer, float and character type of data as
// input.

```
#include <iostream>
using namespace std;

// Template function to swap two values
template <typename T>
void swapValues(T &a, T &b)
{
   T temp = a;
```

```
    a = b;
    b = temp;
}

int main()
{
    // Swapping integers
    int x = 5, y = 10;
    cout << "Before swapping integers: x = " << x << ", y = " << y << endl;
    swapValues(x, y);
    cout << "After swapping integers: x = " << x << ", y = " << y << endl;

    cout << endl;

    // Swapping floats
    float p = 2.5f, q = 4.5f;
    cout << "Before swapping floats: p = " << p << ", q = " << q << endl;
    swapValues(p, q);
    cout << "After swapping floats: p = " << p << ", q = " << q << endl;

    cout << endl;

    // Swapping characters
    char c1 = 'A', c2 = 'Z';
    cout << "Before swapping characters: c1 = " << c1 << ", c2 = " << c2 << endl;
    swapValues(c1, c2);
    cout << "After swapping characters: c1 = " << c1 << ", c2 = " << c2 << endl;

    return 0;
}
```

// output:

```
// Before swapping integers: x = 5, y = 10
// After swapping integers: x = 10, y = 5

// Before swapping floats: p = 2.5, q = 4.5
```

```
// After swapping floats: p = 4.5, q = 2.5

// Before swapping characters: c1 = A, c2 = Z
// After swapping characters: c1 = Z, c2 = A
```

// 12) Write an object-oriented program to implement a generic Number Class that
// can accept either int or float data type and perform basic calculation like +,-,/
// and *.

```
#include <iostream>
using namespace std;

// Template class Number
template <typename T>
class Number
{
private:
   T value;

public:
   // Constructor
   Number(T v)
   {
      value = v;
   }

   // Addition
   Number operator+(const Number &other)
   {
      return Number(value + other.value);
   }

   // Subtraction
   Number operator-(const Number &other)
   {
      return Number(value - other.value);
```

```
    }

    // Multiplication
    Number operator*(const Number &other)
    {
        return Number(value * other.value);
    }

    // Division
    Number operator/(const Number &other)
    {
        if (other.value == 0)
        {
            cout << "Error: Division by zero!" << endl;
            return Number(0); // Or handle it differently
        }
        return Number(value / other.value);
    }

    // Display the value
    void display() const
    {
        cout << value << endl;
    }
};

int main()
{
    // Using with integers
    Number<int> n1(20);
    Number<int> n2(5);

    cout << "Integer calculations:" << endl;
    cout << "n1 + n2 = ";
    (n1 + n2).display();
    cout << "n1 - n2 = ";
```

```
    (n1 - n2).display();
    cout << "n1 * n2 = ";
    (n1 * n2).display();
    cout << "n1 / n2 = ";
    (n1 / n2).display();

    cout << endl;

    // Using with floats
    Number<float> f1(12.5);
    Number<float> f2(2.5);

    cout << "Float calculations:" << endl;
    cout << "f1 + f2 = ";
    (f1 + f2).display();
    cout << "f1 - f2 = ";
    (f1 - f2).display();
    cout << "f1 * f2 = ";
    (f1 * f2).display();
    cout << "f1 / f2 = ";
    (f1 / f2).display();

    return 0;
}
```

// output

```
// Integer calculations:
// n1 + n2 = 25
// n1 - n2 = 15
// n1 * n2 = 100
// n1 / n2 = 4

// Float calculations:
// f1 + f2 = 15
// f1 - f2 = 10
// f1 * f2 = 31.25
```

```
// f1 / f2 = 5
```

// 13) Write an object-oriented program to implement a generic Stack. Incorporate

// all the possible operation on Stack in the program.

```
#include <iostream>
using namespace std;

template <typename T>
class Stack
{
   T *arr;
   int top;
   int capacity;

public:
   Stack(int size = 10)
   {
      capacity = size;
      cout << "Stack created of size " << capacity << endl;
      arr = new T[capacity];
      top = -1;
   }

   ~Stack()
   {
      delete[] arr;
      cout << "Stack destroyed\n";
   }

   void push(T value)
   {
      if (isFull())
      {
         cout << "Stack Overflow! Cannot push " << value << endl;
         return;
```

```
    }
    arr[++top] = value;
}

T pop()
{
    if (isEmpty())
    {
        cout << "Stack underflow! Cannot pop" << endl;
        cout << "Returning default value of T: " << T() << endl;
        return T();
    }
    return arr[top--];
}

T peek()
{
    if (isEmpty())
    {
        cout << "Stack is empty !" << endl;
        return T();
    }
    return arr[top];
}

void display()
{
    if (isEmpty())
    {
        cout << "Stack is empty !" << endl;
        return;
    }
    cout << "\nStack elements:\n";
    for (int i = 0; i <= top; i++)
    {
        cout << arr[i] << " ";
```

```
        }
        cout << endl;
    }

    bool isEmpty()
    {
        return top == -1;
    }

    bool isFull()
    {
        return top >= capacity - 1;
    }
};

int main()
{
    Stack<char> s3(5);

    s3.display();
    cout << "Popped: " << s3.pop() << endl;

    s3.push('c');
    s3.push('a');
    s3.push('m');
    s3.push('p');
    s3.push('l');
    s3.push('x');

    s3.display();

    cout << "Top element: " << s3.peek() << endl;

    cout << "Popped: " << s3.pop() << endl;

    s3.display();
```

```
    return 0;
}
// output:
// Stack created of size 5
// Stack is empty !
// Stack underflow! Cannot pop
// Returning default value of T:
// Popped:
// Stack Overflow! Cannot push x

// Stack elements:
// c a m p l
// Top element: l
// Popped: l

// Stack elements:
// c a m p
// Stack destroyed
```

// 14) Write a generic function that will sort a character string, integer and float
// value. Create a menu with appropriate options and accept the values from the
// user.

```
#include <iostream>
using namespace std;

// Template function to sort array or string
template <typename T>
void sortValues(T arr[], int n)
{
    // bubble sort
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
```

```
            {
                // Swap
                T temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Function to display the array or string
template <typename T>
void displayValues(T arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main()
{
    int choice;
    do
    {
        cout << "\n--- Sorting Menu ---\n";
        cout << "1. Sort a character string\n";
        cout << "2. Sort integer values\n";
        cout << "3. Sort float values\n";
        cout << "0. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
```

```
    char str[100];
    cout << "Enter a string: ";
    cin >> str;
    int n = 0;
    while (str[n] != '\0')
        n++;
    sortValues(str, n);
    cout << "Sorted string: ";
    displayValues(str, n);
}
else if (choice == 2)
{
    int n;
    cout << "Enter number of integers: ";
    cin >> n;
    int arr[100];
    cout << "Enter integers:\n";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    sortValues(arr, n);
    cout << "Sorted integers: ";
    displayValues(arr, n);
}
else if (choice == 3)
{
    int n;
    cout << "Enter number of floats: ";
    cin >> n;
    float arr[100];
    cout << "Enter floats:\n";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
```

```
        sortValues(arr, n);
        cout << "Sorted floats: ";
        displayValues(arr, n);
    }
    else if (choice == 0)
    {
        cout << "Exiting...\n";
    }
    else
    {
        cout << "Invalid choice!\n";
    }

  } while (choice != 0);

  return 0;
}
```

// output

```
// --- Sorting Menu ---
// 1. Sort a character string
// 2. Sort integer values
// 3. Sort float values
// 0. Exit
// Enter your choice: 1
// Enter a string: xzyacb
// Sorted string: a b c x y z

// --- Sorting Menu ---
// 1. Sort a character string
// 2. Sort integer values
// 3. Sort float values
// 0. Exit
// Enter your choice: 2
// Enter number of integers: 4
```

```
// Enter integers:
// 23
// 6
// 3
// 2
// Sorted integers: 2 3 6 23

// --- Sorting Menu ---
// 1. Sort a character string
// 2. Sort integer values
// 3. Sort float values
// 0. Exit
// Enter your choice: 3
// Enter number of floats: 3
// Enter floats:
// 1.2
// 2.3
// 0.2
// Sorted floats: 0.2 1.2 2.3

// --- Sorting Menu ---
// 1. Sort a character string
// 2. Sort integer values
// 3. Sort float values
// 0. Exit
// Enter your choice: 0
// Exiting..
```

// 15) Write a template function called find(). This function searches an array for an

// object. It returns either the index of the matching object (if one is found) or

// -1 if no match is found.

```
#include <iostream>
#include <string>
```

```
using namespace std;

// Template function to find an element in the array
template <typename T>
int find(T arr[], int n, T target)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == target)
        {
            return i; // Return index if found
        }
    }
    return -1; // Return -1 if not found
}

// User-defined class Person
class Person
{
public:
    string name;
    int age;

    Person() : name(""), age(0) {}
    Person(string n, int a) : name(n), age(a) {}

    // Overload equality operator to compare Person objects
    bool operator==(const Person &other) const
    {
        return (name == other.name && age == other.age);
    }
};

int main()
{
    cout << "Searching in arrays of different types:" << endl;
```

```
// Integer array
int intArr[] = {10, 20, 30, 40};
int intSize = 4;
int intTarget = 30;
cout << "\nSearching for " << intTarget << " in integer array...\n";
int index = find(intArr, intSize, intTarget);
if (index != -1)
   cout << "Found at index " << index << endl;
else
   cout << "Not found" << endl;

// Float array
float floatArr[] = {1.1, 2.2, 3.3, 4.4};
int floatSize = 4;
float floatTarget = 3.3;
cout << "\nSearching for " << floatTarget << " in float array...\n";
index = find(floatArr, floatSize, floatTarget);
if (index != -1)
   cout << "Found at index " << index << endl;
else
   cout << "Not found" << endl;

// Character array
char charArr[] = {'x', 'y', 'z'};
int charSize = 3;
char charTarget = 'y';
cout << "\nSearching for '" << charTarget << "' in character array...\n";
index = find(charArr, charSize, charTarget);
if (index != -1)
   cout << "Found at index " << index << endl;
else
   cout << "Not found" << endl;

// Array of objects
Person people[] = {
```

```
        Person("Darshan", 25),
        Person("dp", 30),
        Person("kp", 35)};
    int peopleSize = 3;
    Person target("dp", 30);
    cout << "\nSearching for Person (dp, 30) in Person array...\n";
    index = find(people, peopleSize, target);
    if (index != -1)
        cout << "Found at index " << index << endl;
    else
        cout << "Not found" << endl;

    return 0;
}
```

// output:

```
// Searching in arrays of different types:

// Searching for 30 in integer array...
// Found at index 2

// Searching for 3.3 in float array...
// Found at index 2

// Searching for 'y' in character array...
// Found at index 1

// Searching for Person (dp, 30) in Person array...
// Found at index 1
```